Thesis

Tom Zurales

December 2025

Abstract

This thesis implements and analyzes a novel, viewpoint-aware method for map point culling in keypoint-based visual SLAM systems. This method makes use of a K-Nearest Neighbors Observability Model to estimate viewpoint-dependent observability for each map point using constant additional space per point. This estimate supports continuous calculation of a map point's existence probability using a Bayesian update step implemented by an Existence Probability Estimator. This thesis explores the use of viewpoint-based existence estimations as a method of culling outdated map points, such as those originally seen on an object which has since moved, and as an extension to the RANSAC algorithm to prioritize map points with higher existence probability. The implementation is released as an open-source library, libVBEE, along with an ORB-SLAM3 integration utilizing the library for point culling and RANSAC improvement. Additionally, a set of stereo visual datasets are provided, containing scenarios specifically intended to exercise and characterize the performance of the system. Effects on standard non-dynamic SLAM datasets are discussed alongside the new datasets.

Contents

Definitions **7**

1 Introduction **9**

1.1 Problem Context 9

1.2 Problem Statement 10

1.3 Objectives and Scope 13

1.4 Research Questions 16

1.5 Contribution 17

1.5.1 The VBEE Library 17

1.5.2 ORB-SLAM3 Integration 17

1.5.3 Episodic Change Datasets 17

1.6 Road Map 18

2 Background **19**

2.1 Keypoint-Based Visual SLAM 19

2.1.1 Image Features 20

2.1.2 Robust Estimation and RANSAC 20

2.1.3 Tracking and Mapping 22

2.1.4 Loop Closure and Relocalization 23

2.1.5 Map Reuse 24

2.2 ORB-SLAM3 25

3 Related Work **26**

3.1 Place Recognition over Long Time Horizons 26

3.2 Change Detection for Map Maintenance 27

3.3 Dynamic Rejection 28

3.4 Lifelong SLAM 28
3.5 Map Point Observability 28

4 Implementation **29**
4.1 Definitions and Functions 29
4.2 Method Overview 31
4.2.1 The Existence Estimation Framework 31
4.2.2 The Observability Model 32
4.2.3 Existence Probability Estimator 33
4.2.4 Overall Observability 34
4.3 System Implementation 35
4.3.1 Existence Estimation Framework Implementation 35
4.3.2 KNN Observability Model 36
4.3.3 Existence Probability Estimator 38
4.4 ORB-SLAM3 Integration 39
4.4.1 Culling Logic Implementation 41
4.5 Dataset Creation 41
4.6 Observability Scenarios Dataset 42
4.6.1 Episodic Visual SLAM Dataset 43
4.7 Parameter Optimization 43
4.8 Experimental Setup 49

5 Results and Analysis **50**
5.1 EPE Analysis and Optimization 50
5.2 Evaluation Metrics 50
5.3 SLAM System Configurations 50
5.4 Parameter Optimization 50
5.4.1 Existence Estimation Framework Parameter Selection 50

5.5 Results 52
5.5.1 Quantitative Evaluation 52
5.5.2 Qualitative Evaluation 52
5.5.3 Ablation Study 52

6 Conclusion and Future Work **53**
6.1 Future Work 53

References **54**

Appendices **60**
6.2 Key Algorithm Implementations 60
Key Algorithm Implementations 60
Full Results 61

List of Figures

1 OMR Method Comparison 13

2 SLAM Family Tree 19

3 Image Feature Extraction and Matching 21

4 2D RANSAC Example 22

5 Loop Closure Example 23

6 Simplified ORB-SLAM3 Operational Diagram 25

7 Existence Estimation Framework 31

8 2D Global Observability 33

9 2D cutaways of examples from the Observability Scenarios Dataset 42

10 Example images from each trajectory of the episodic visual SLAM dataset . 43

11 Damping Coefficient Selection Results. 52

List of Algorithms

1 VBEE Initialization 35

2 VBEE Update 36

3 KNN Observability Estimation 37

4 KNN Model Update 38

5 Bayesian Update Step 38

6 Find Map Points in Camera View 40

7 Update Existence Probabilities 40

8 Generate Observability Scenario 42

9 Objective Function: $n_{converge}$ 45

10 Objective Function: $\epsilon_{converge}$ 46

11 Objective Function: t_{update} 47

12 Observability Model Objective Function 48

Definitions

1 Introduction

The research presented in this thesis addresses a challenge common in long-term usage of SLAM systems: the need to continuously localize and map in non-static environments. This thesis presents a novel, lightweight method for culling outdated map points from previously generated maps using a probabilistic, viewpoint-aware model of map point observability. This method is designed to operate efficiently on mobile robotics platforms with limited computational resources. The effectiveness of this approach is demonstrated in identifying outdated map points, improving localization accuracy, and improving the robustness of SLAM between different runs of the system.

1.1 Problem Context

The International Space Station (ISS) has three robots on board known as Astrobees. These robots were launched with the intention of being a platform for zero-gravity robotics research, a camera rig for recording astronaut activity, and a sensor platform for performing station keeping and monitoring tasks [1]. Core to each of these missions is the ability for Astrobee to autonomously localize and navigate throughout the U.S. Orbital Segment (USOS). Unfortunately, the harsh realities of navigation on board the ISS quickly became apparent. The high-level structure of the station is constant, but the visual features within the station are not stable, changing due to equipment resupplies, hardware movements, astronaut activity, and lighting changes. The irregularity in the visual features of the ISS degraded the performance of the Astrobee localizer, which relied on visual features stored in a pre-generated map [2]. Mapping the ISS required input from ground crew and astronauts, which, due to the tight timetables associated with ISS operations, led to infrequent map updates. These outdated maps caused frequent lost states, inaccurate localization estimations, and overall poor performance of the navigation stack. Eventually, Astrobee activities were restricted almost entirely to the Japanese Experiment Module (JEM), a relatively small area which

could be kept visually stable, and could be quickly remapped when needed [3].

A project was proposed to augment Astrobee's navigation stack with a keypoint-based visual SLAM (KV-SLAM) system. The idea behind this change was to have the SLAM system load an initial map from the previous mission, localize within this map, then update the map with the latest visual features from the environment. At the end of the operation, the updated map could be saved off for use in the next mission [4]. For this research, this operational concept of iteratively updating previous maps between operations is referred to as *episodic SLAM*. Testing of the SLAM system presented the same issues encountered by the Astrobee localizer when attempting to reuse outdated maps. The conjecture stemming from these results was that for reused maps to be useful, they must not only be updated with the latest visual features, but must also remove features which have become outdated.

Three key lessons were learned from the Astrobee work which motivated the research presented here. First, the episodic operating model has potential in a wide variety of applications, providing a method for regular, large-scale environmental monitoring of changing environments such as factories, warehouses, and offices. Second, map reuse is a powerful tool for robotic systems operating in the same environment over long periods of time, allowing for the preservation of structural map data and the global reference frame. Finally, SLAM systems are not inherently robust to outdated maps, showing degraded performance as the initial map diverges from the true environment.

1.2 Problem Statement

A future use case of episodic KV-SLAM would be for operations in environments that are intermittently occupied, such as workplaces. The benefit of this method is that the robot can avoid operating during highly dynamic times, with the trade-off being that the environment may be significantly changed between sessions of operation. In the case of Astrobee, the navigation system struggled to reconcile inconsistencies between the internal map and the real world, regularly requiring astronaut assistance to recover. Trust in autonomous systems

is already precarious, quickly falling upon the observation of failure [5]. Robotic SLAM failures can lead to navigational instability, task failure, and even the possibility of physical damage to the surrounding environment or the robot itself [6]. Therefore, in order for autonomous SLAM-based robotic navigation to see widespread use, it must be accurate in its estimations and capable of withstanding a wide variety of failure cases. This research focuses on reducing failures caused by the existence of outdated map features in reused maps during episodic KV-SLAM operations. For simplicity, the topic of reusing previously generated maps that have gone out of date is referred to as outdated map reuse (OMR).

There are three critical issues that appear in systems that do not handle the challenges of OMR sufficiently well. The first relates to the size of the map, as only adding newly observed features without removing outdated ones results in a monotonically increasing map size. This puts an ever-increasing strain on compute resources to deal with the large map and increases the complexity of optimization steps. The second issue concerns relocalization, which is the process by which a KV-SLAM system redetermines its position within the map after being lost. Relocalization starts by identifying candidate positions within the map that are visually similar to the current camera image. A high proportion of outdated map points in the map interferes with this candidate selection, as the map's understanding of the visual features of the environment no longer matches reality. The final issue affects the RANSAC process, a function that is run regularly as part of determining the 3D transformation between successive frames. The RANSAC algorithm selects a random sample of points from the map and the new image to be localized and attempts to determine a consistent 3D transformation that explains the correspondence. High proportions of outdated map points increase the risks of the RANSAC algorithm failing to find a consistent pose for the new frame, even if one exists. The map representation, relocalization, and RANSAC will be covered in greater detail in section 2.1.

All of these issues with OMR stem from the existence of bad data in the KV-SLAM map. There are two ways to prevent such bad data from affecting the map. The first option

is to proactively prevent map features with potential to become outdated (i.e., located on an object that can move) from ever entering the map. In this case, the map will never be outdated because no features in the map can move. The second option is to retroactively identify and remove outdated features from the map once it has been loaded for reuse. The mechanism at play in the proactive case tends to be semantic identification and segmentation of input images, allowing semantic labels to determine whether map points that fall on certain objects are kept or not. These semantic processes require significant compute and storage resources, often making them unsuitable for compute-constrained applications such as small-scale mobile autonomous vehicles (MAVs). In the reactive case, the methods stray away from semantics and towards change detection. Change detection identifies inconsistencies between the map and the camera observations, flagging points for removal once confidence in their existence drops below some threshold. However, a gap can be noticed in the change detection methods available today. This operation of removing map points based on some criteria comes up frequently in SLAM, and will be referred to as map point removal (MPR) throughout this research. While change detection methods do collect point-level metadata, to the author's knowledge no change detection methods collect metadata on the global observability of each map point, leaving today's systems susceptible to overzealously flagging map points that are temporarily obstructed from certain viewpoints for removal. The semantic and change-detection methods will be discussed in greater detail in section 3.

The key takeaway is that OMR can greatly benefit episodic KV-SLAM systems, but without proper handling of outdated data, OMR utilization creates new failure cases that degrade the system's performance. The methods available that allow OMR to be successfully integrated make a trade-off between computational requirements, storage requirements, and effectiveness as illustrated in Figure 1. As an engineering problem, the constraints that drive the decision of methodology for MPR are size, weight, power, and cost. It is not always possible to simply use a better computer, and in those cases, when accurate outdated MPR is needed, the available methods can fall short of requirements. The conjecture of this

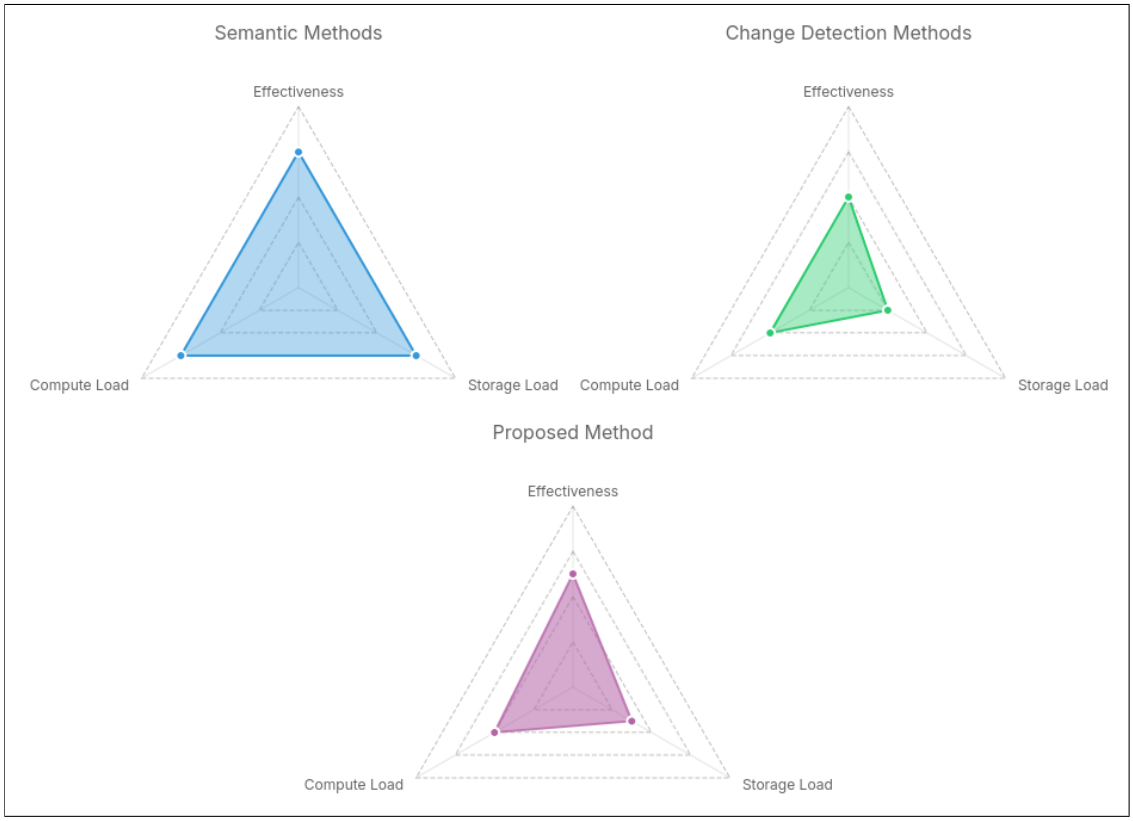

Figure 1: A qualitative comparison of compute load, storage load, and effectiveness between various OMR point removal methods.

research is that a novel change detection method that models the global observability of map features at the point level could increase the accuracy of outdated map point identification without a significant increase in computational power requirements.

1.3 Objectives and Scope

The research presented in this thesis was inspired by and is limited in scope to map reuse in episodic KV-SLAM operations. Limiting the scope does mean that some relevant topics will not be explored in depth, but will be mentioned in section 6.1 as potential areas for follow-on research. The justification for the scope selection is given here.

The methods presented in this research will not work for all SLAM systems, or on all SLAM-generated maps. While change detection techniques do exist for dense maps, the method presented here would not be suitable, as it relies on locating the same map features

at different points in time. Dense maps identify matching positions through geometric correspondence, as opposed to feature descriptors. This applies to LiDAR-based SLAM systems as well, as no descriptor is generated for each point. The method presented here relies on the capability to apply and persist metadata to a particular map point, something that is not possible without point descriptors, leaving KV-SLAM as the only option for this work.

The focus on map reuse and the episodic operating model can be motivated by the use cases discussed previously. On the ISS, the ideal navigation system would be a self-updating localizer rather than a SLAM system. The primary difference is that a localizer utilizes the same map and reference frame for all operations, while a SLAM system generates and uses its own map. SLAM's features are extremely useful for exploration of unknown areas, but less so for repeated operations in known areas. The combination of map reuse and the episodic operating model allows the system to act as a self-updating localizer, maintaining a map that is up-to-date with the last observed state of the world while preserving as much previously collected data as possible. Within this scope, the objectives of this research can be defined.

This research has two primary objectives. The first is the implementation of a novel viewpoint-based MPR method that considers the historical, point-level, viewpoint-dependent observability of map points in the decision to keep or discard points. The second is the integration of this MPR method into an existing KV-SLAM system. Both objectives have several sub-goals, which are detailed below.

Objective 1: Implementation of a viewpoint-based MPR method

Goal 1: Speed

As this system is intended to be run as part of real-time SLAM operations, no function performed by the model can take longer than the delta time between image frames. Ideally, each function should be significantly faster than this to allow for all SLAM system operations to complete as well.

This goal also refers to the speed of identifying outdated map points, meaning a point that is outdated should be identified and culled with a minimal number of observations.

Goal 2: Size

To run well on resource-constrained hardware, additional space requirements (RAM and disk) must be kept low. The space complexity of the system should be constant, and not a function of the number of map points or observations.

Goal 3: Accuracy

The system must only identify points as outdated once sufficient evidence of them being out of date has been collected. The system should be robust to falsely identifying active map points as outdated and outdated map points as active.

Objective 2: Integration into an existing KV-SLAM system

To verify the performance and effectiveness of the viewpoint-based MPR method, it must be implemented into an existing SLAM system and benchmarked in various scenarios.

Goal 1: Maintain performance in the general case

This research is focused on outdated map reuse and therefore should not affect the performance of the SLAM system when not reusing an outdated map. The modified system should perform identically to the base system in this case.

Goal 2: Improve performance when using outdated maps

To be considered successful, the SLAM implementation should perform better than the base system when using an outdated initial map. The criteria for better performance are defined in section 5.2.

Goal 3: Maintain computational requirements

If the goals of Objective 1 are met, integration of the MPR method should not cause the computational requirements of the system to increase significantly.

With these objectives in hand, a set of questions can now be defined which, when answered, will determine whether the goals of the objectives have been successfully achieved.

1.4 Research Questions

Question 1: How can the historical viewpoint-dependent observability of points be modeled in order to meet the objectives laid out in Objective 1?

Integrating historical viewpoint-dependent observability into existing change detection approaches requires a compact model of observability at the point level. Three model implementations are explored in section 4.3.2 and evaluated in section 5.

Question 2: How does the integration of historical observability data affect the estimation of point existence confidence?

Each of the three model implementations is integrated into an existing change detection model, discussed in section 4.3.3, which utilizes a Bayesian update step to iteratively update a confidence estimation of the map point's existence. Each of the three models is evaluated for speed, accuracy, and robustness in section 5.

Question 3: Can the change detection model be integrated into an existing KV-SLAM system in a way that meets the goals of Objective 2?

The extended confidence update is integrated into ORB-SLAM3, a popular KV-SLAM system, for evaluation against a series of datasets intended to test the system across a range of simple and challenging situations. The integration is discussed in section 4.4, and the

evaluation is presented in section 5.

1.5 Contribution

Through this research, three contributions are made to the subject of long-term map reuse in KV-SLAM. First, an open-source library is released which provides reusable access to the mathematical implementation of the VBEE framework. Next, a modified version of ORB-SLAM3 with the VBEE framework integrated is released for ease of further testing and research. Finally, a collection of stereo visual datasets is provided covering situations particularly challenging for intermittently run KV-SLAM systems, enabling demonstration and comparison of the system's capabilities.

1.5.1 The VBEE Library

The VBEE library (libVBEE) is a C++ implementation of the viewpoint-based existence estimation framework, with hooks for simple integration into any existing KV-SLAM implementation.

1.5.2 ORB-SLAM3 Integration

ORB-SLAM3, an extremely popular KV-SLAM implementation, is modified to utilize libVBEE for estimating the probability of map point existence within the global map. These probabilistic estimations are then used in the selection of keypoints for culling operations, regularly removing points which drop below a certain threshold, and for improving relocalization operations through an updated RANSAC implementation which weights high-probability map points above low.

1.5.3 Episodic Change Datasets

A set of stereo visual datasets is provided, each accompanied by a map generated using an unmodified implementation of ORB-SLAM3.

1.6 Road Map

Chapter 2 provides a high-level overview of the concepts core to this research, including the inner workings of KV-SLAM, relocalization, RANSAC, and map point culling.

Chapter 3 digs deeper into research related to improving long-term SLAM operations and detecting and removing extraneous data from SLAM maps.

Chapter 4 discusses the implementation details of a viewpoint-based historical observability model, the creation of a point removal system that implements this model, and its integration into ORB-SLAM3. Additionally, the hardware and software supporting the collection of the stereo visual datasets are described.

Chapter 5 covers the tests performed on the observability model, the existence probability estimator, and the integrated SLAM system. Tests are performed in simulation to establish initial parameters and on generated datasets to compare real-world and simulated performance.

Chapter 6 aggregates the results from Chapter 5 and evaluates them against the defined objectives of this research to determine whether this method is an effective solution to the problems discussed so far. Areas requiring further study are also discussed.

2 Background

This chapter provides a high-level overview of SLAM, with particular focus on the KV-SLAM modality and the ORB-SLAM3 implementation.

2.1 Keypoint-Based Visual SLAM

SLAM is the joint problem of simultaneously generating a map of an environment and estimating the position of an observer within that map based on sensor observations. First explored in the 1980s [7], SLAM has become a de facto standard in robotics for tasks that require operations within unfamiliar environments. A wide variety of sensors can be utilized by SLAM, the most common being LiDAR, cameras, and RGB-D sensors. The high-level relationships between these sensor modalities are shown in Figure 2. Keypoint-based visual SLAM refers to a subset of the wider SLAM ecosystem characterized by the use of cameras as the primary sensor, and image features extracted from images as the primary data for tracking and mapping.

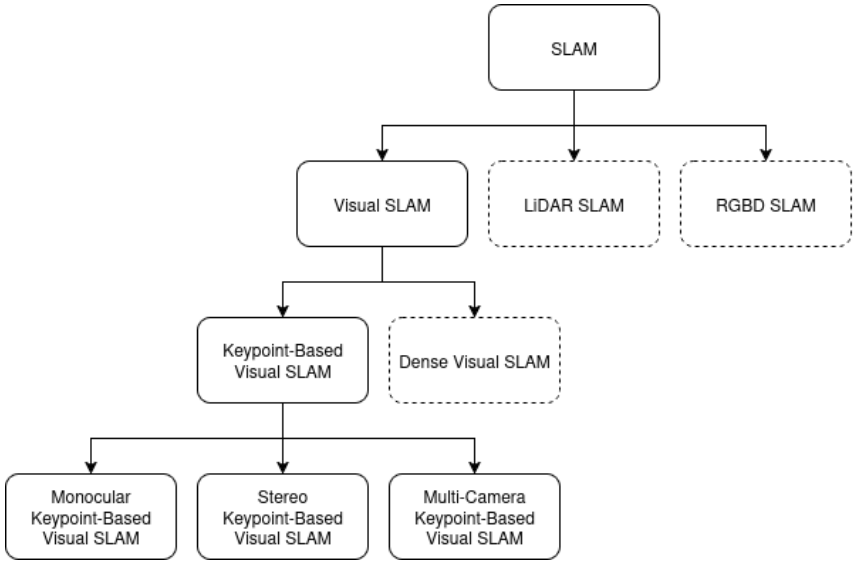

Figure 2: Overview of the relationships between popular SLAM modalities.

The first implementations of KV-SLAM appeared in the early 2000s [8][9], making the

field relatively young. This can be attributed to the difficulty of estimating 3D geometries from 2D images, unlike LiDAR and RGB-D which are capable of direct 3D environmental measurements. However, due to the extremely low cost and wide availability of camera sensors, KV-SLAM has become a popular modality for robotics, AR/VR, and autonomous driving applications. This research ties into the structures and inner workings of KV-SLAM systems, so an overview of the concepts and techniques used in KV-SLAM is provided below.

2.1.1 Image Features

An image feature is the combination of a keypoint and a feature descriptor [10]. Keypoints are locations in an image designed to be invariant to lighting, scale, translation, and rotation, while descriptors are a data vector that acts as a unique fingerprint for a feature. Keypoints are located, and their descriptors generated, in a process called feature extraction.

Image features can be matched between frames using their descriptors. For many features such as SIFT [10], SURF [11], and ORB [12], the descriptor is a vector of floating-point values, calculated from pixel values surrounding the keypoint, which can be compared using a distance metric such as Euclidean distance or Hamming distance.

The purpose of feature extraction is to reduce the large input image data to a small set of easily detectable and trackable features, allowing the majority of the image data to be discarded. Figure 3 shows the output of the ORB feature extractor on two input images, and the matching features between the two images. The ability to match the same set of points across multiple viewpoints is core to how 3D map points are estimated from multiple 2D images.

2.1.2 Robust Estimation and RANSAC

Image feature matching is a noisy process, meaning the correspondences between features in different images will not always be correct. This is especially true when the environment contains repetitive patterns. To address this, robust estimation techniques are used to filter

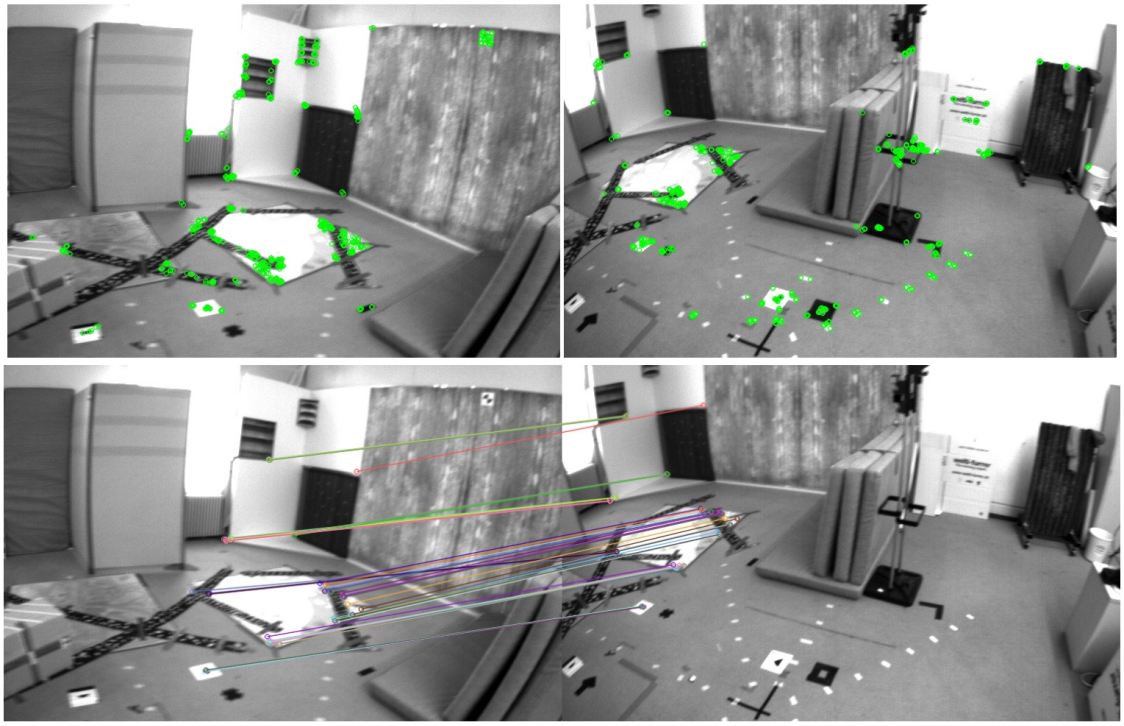

Figure 3: (Top) ORB features extracted from two frames of the EuRoC MAV V1_03 dataset [13]. (Bottom) Corresponding ORB feature matches between the two frames.

outliers from the data. In the context of KV-SLAM, the most common robust estimation technique is Random Sample Consensus (RANSAC) [14].

RANSAC is an iterative algorithm that estimates model parameters from a dataset containing outliers. It works by randomly selecting a subset of the data, fitting the model to this subset, and then counting the number of inliers that fit the model within a certain threshold. The process is repeated until a model is found that matches a sufficient number of inliers, or a maximum number of iterations is reached.

Figure 4 provides a visual representation of the RANSAC process. If one were to sample linear data with a noisy sensor that sometimes produces outliers, the output may look like the left-most plot of the figure. As can be seen, the best-fit line is severely skewed by the outliers. By iteratively selecting a random sample of the input data, fitting the model to this sample, and counting the number of inliers, RANSAC can find a solution that explains a high proportion of the observations.

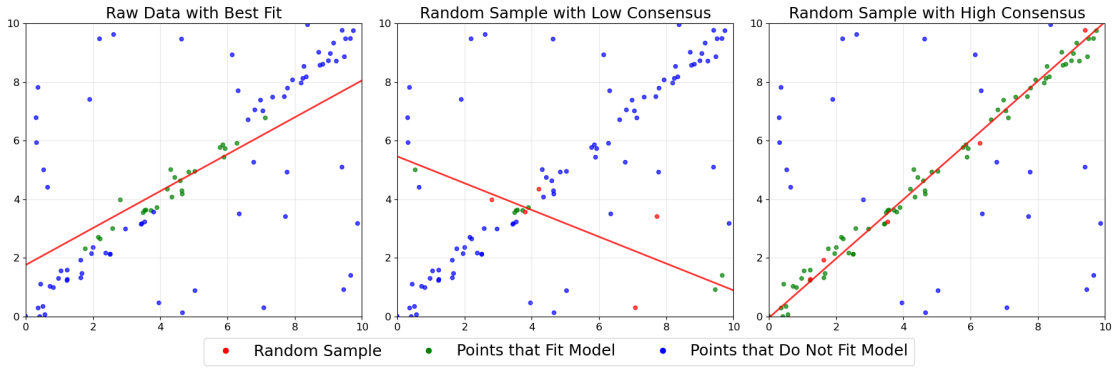

Figure 4: A demonstration of how RANSAC determines model parameters through iterative random sampling and model application.

It should be noted that RANSAC requires a low proportion of outliers in the data to be effective, as it relies on randomly selecting a set of inliers for model fitting. For that reason, preprocessing steps that reduce the number of potential outliers in sample data are useful for improving the speed and effectiveness of RANSAC.

2.1.3 Tracking and Mapping

In the context of KV-SLAM, tracking refers to the process of estimating the real-time position and orientation of the camera, while mapping refers to the process of generating a 3D map of the environment. Methods such as the eight-point algorithm can be used to simultaneously calculate the transformation and depths between two images taken from different viewpoints using their 2D-2D feature correspondences [15][16]. This is primarily used for map initialization, before any 3D map points have been established. Once a set of 3D map points has been established, tracking can be achieved using the Perspective-n-Point (PnP) algorithm [14], with mapping utilizing triangulation to generate new map points [9].

Incorrect 2D-2D and 2D-3D correspondences degrade the performance of the tracking and mapping processes. To account for outliers in the correspondences, RANSAC is used to estimate the transformations and 3D depths on a subset of the matches with the goal of finding a transformation that maximizes the number of correspondence inliers.

In the context of this research, the 2D-3D correspondences between an image and the

map may contain more outliers when the map contains a significant number of outdated map points. If this conjecture is correct, an improvement in the speed and robustness of the RANSAC process should be observed after the implementation of the proposed viewpoint-based MPR method.

2.1.4 Loop Closure and Relocalization

Loop closure is a technique used to correct drift during large-scale KV-SLAM operations [17]. The process works by recognizing when the system has entered a previously visited place, usually by detecting visual similarity between this place and the current image. Due to noise inherent to the system, the current position estimate may not match the position estimate from the previous visitation. The drift error is the difference between the previous and current position estimates, and can be propagated through the historical trajectory estimations, re-aligning the two positions. Figure 5 shows how loop closure identifies a previously visited place and corrects the trajectory to remove accumulated drift.

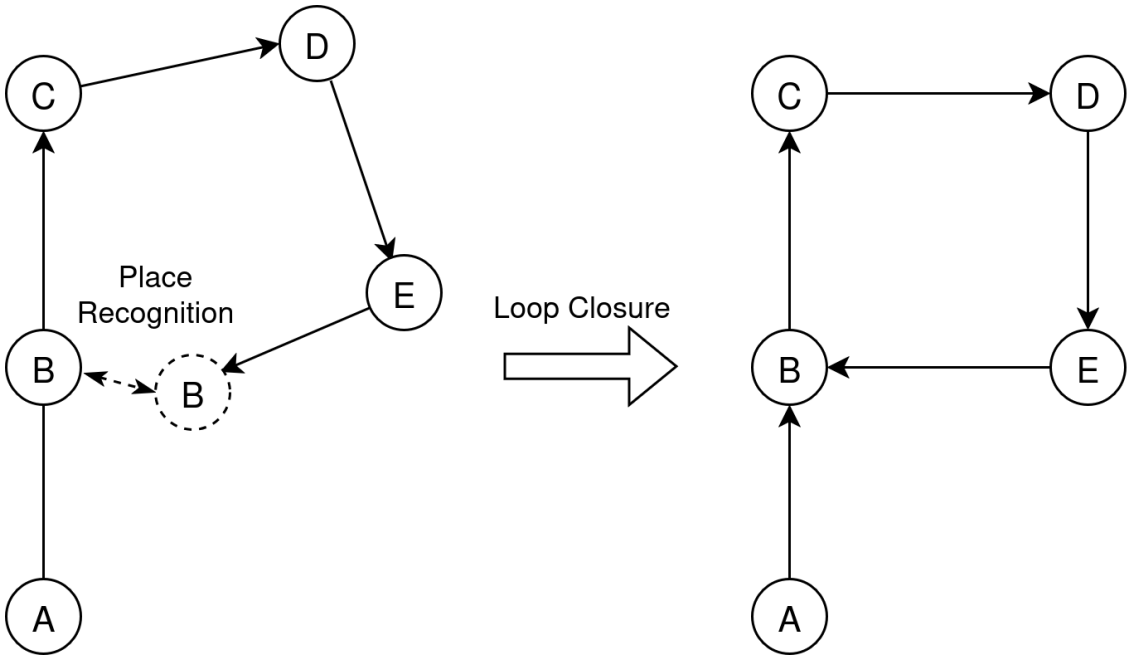

Figure 5: An example of how loop closure can correct drift by constraining the trajectory based on detected loops.

Relocalization is used when the SLAM system is "lost," meaning a map of the environ-

ment exists but the location of the camera within this map is unknown. This can happen for many reasons such as traversing an area with low visual texture, dim lighting, or temporary obstruction of the camera by a moving object. To recover, the SLAM system looks for positions within the map that are visually similar to the current image. If a strong candidate position is found, the system attempts to find a valid pose for the image in the map using PnP and RANSAC.

Both of these operations rely on the ability to recognize visual similarity between the current image and the map. As the image data is discarded, visual similarity must be recognized another way. In KV-SLAM systems, this is commonly accomplished using a bag-of-words model, which represents the map's feature descriptors as a collection of visual words, allowing the system to recognize previously visited places based on the presence of these words in the current image's feature descriptors [18].

For this research, the existence of outdated map points in the map may cause the bag-of-words model to produce false positives or negatives, leading to incorrect relocalization or loop closure detections. Removing these outdated features from the map and from the bag-of-words model should improve the performance of these operations, as the system will no longer attempt to match against outdated features.

2.1.5 Map Reuse

Map reuse is core to this research, as it is what allows SLAM to operate in the episodic operating model. By loading a previously generated map into the SLAM system, the system can operate in a known coordinate frame and maintain a consistent map of the environment. In the episodic operating model, the system is expected to operate in the same environment over long periods of time. By reusing a previously generated map, updating it with new observations, and removing outdated observations, the system can maintain a map that is up-to-date with the last observed state of the world while preserving as much previously collected data as possible.

2.2 ORB-SLAM3

ORB-SLAM3 is a KV-SLAM implementation that is popular in research contexts due to its performance, open-source codebase, and use of methods and techniques that are well documented and widely understood. Figure 6 shows a simplified diagram of the internal processes of ORB-SLAM3. The system supports a variety of input modalities including monocular and stereo cameras, RGB-D cameras, and IMU data. It splits work across multiple threads, allowing it to perform tracking, mapping, loop closure, relocalization, and optimization concurrently.

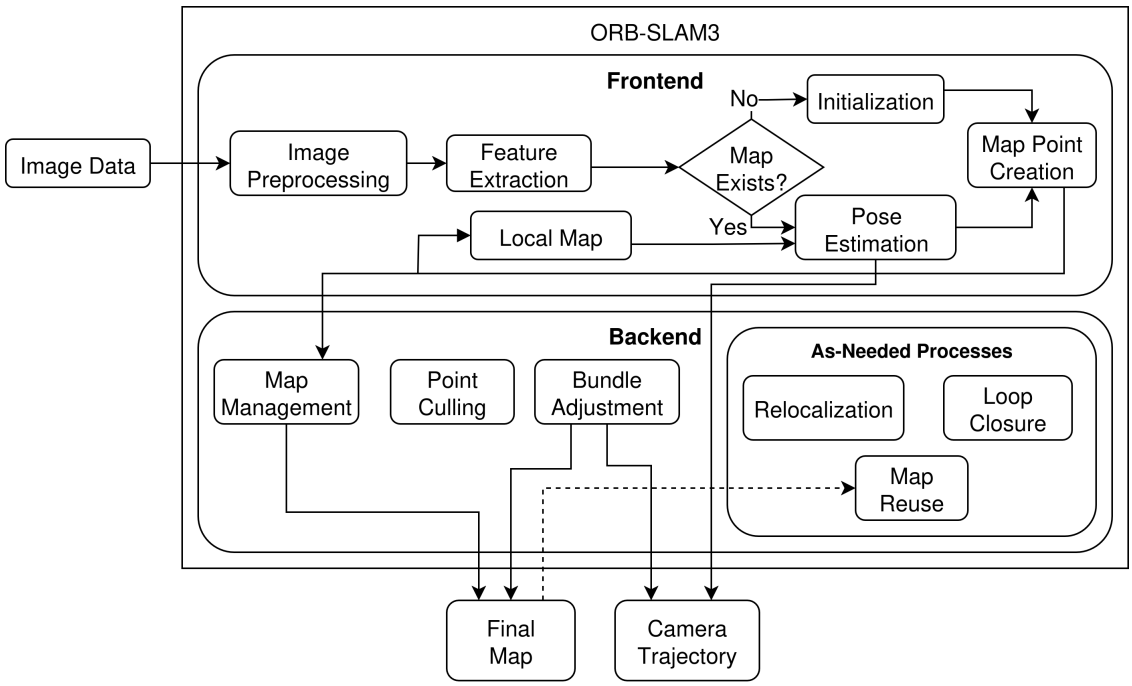

Figure 6: A simplified diagram of the inputs, outputs, and subprocesses of ORB-SLAM3.

3 Related Work

This section contextualizes the proposed research within the larger body of SLAM research. At its core, this thesis falls under the umbrella of lifelong SLAM. Lifelong SLAM has varied definitions throughout the literature, but for simplicity, this research adopts the definition provided by Shi et al. [19], describing lifelong SLAM as the ability for a robot to generate, maintain, and localize within a map of a particular environment over an extended period of time. This is in contradistinction to the standard SLAM operating model, which tends to require short timeframes due to the assumption of a static environment. Unlike standard SLAM, lifelong SLAM is expected to operate despite environmental changes such as moved objects, lighting changes, or dynamic objects within sensor view. This research falls under the topic of map maintenance, but has downstream effects on other operations related to lifelong SLAM.

Several research topics fall under lifelong SLAM, with those relevant to the work presented in this thesis discussed below. This research directly impacts the KV-SLAM map, so the downstream effects on other aspects of lifelong SLAM performance will be discussed at the end.

3.1 Place Recognition over Long Time Horizons

As previously discussed, place recognition refers to the ability to determine when an agent has entered a previously visited area. In KV-SLAM, this is commonly achieved using a bag-of-words approach over binary feature descriptors extracted from camera images [20]. However, these features can vary significantly over time due to changes in lighting, weather, scene layout, or viewpoint. Chen et al. [21] proposed using deep learning to identify features that are invariant to conditions and viewpoint, but they are still sensitive to geometric changes.

Milford et al. improved robustness to time-of-day and weather variation by matching

sequences of whole-image comparisons rather than relying on a single image or feature-based matches. However, this approach requires alignment between the image sequence and the reference trajectory, effectively requiring similar speed and direction of travel. Naseer et al. [22] relaxed this constraint by computing a full probability distribution over possible locations, removing the need for trajectory alignment. Both of these approaches are sensitive to viewpoint variation, and can fail when the camera trajectory differs significantly from previously observed trajectories.

Learning-based methods like NetVLAD [23] assign global descriptors to places rather than features, increasing robustness to viewpoint, conditions, and dynamic clutter. HF-Net [24] extends this idea by utilizing global descriptors for large-scale place recognition and local learned features for refinement. The negative aspects of learning-based approaches tend to be their computational load, requiring significantly higher capability than methods that do not require a neural network.

While the methods discussed here do allow for place recognition to succeed despite significant visual and geometric changes, in general place recognition is most effective when the internal scene representation matches the true environment. To accomplish this, the system must be able to identify and correct discrepancies between the map and the current sensor readings.

3.2 Change Detection for Map Maintenance

Change detection in SLAM refers to identifying discrepancies between current sensor observations and the existing map. This functionality supports operations like relocalization and place recognition by acting as the mechanism through which outdated map data is identified so it can be updated. The implementation of change detection depends heavily on sensor modality and map representation.

Girardeau-Montaut et al. [25] implemented point cloud change detection for laser scans of industrial sites using iterative closest point (ICP) [26, 27] to align scans, with Hausdorff

distance [28] used to identify areas of change. Lin et al. [29] extended this to stereo visual SLAM in their system PPCA-VINS, which detects changes between a prior point cloud and current point measurements. The method of aligning point clouds and measuring disparity is effective, but requires both point clouds to be of similar scale and density to achieve optimal alignment. Additionally, these methods do not attempt to model global point observability, meaning features that still exist could be erroneously deleted.

Many modern methods utilize a learning-based approach to align and detect changes in dissimilar data types. Larsen et al. [30]

3.3 Dynamic Rejection

Dynamic rejection is the process of identifying dynamic objects within a scene and rejecting image features that fall on them. This requires either tracking object motion to verify the object is dynamic, or performing image segmentation and semantic identification to categorize objects as either dynamic or static. It can also be accomplished without strict object identification using methods that detect movement in images (e.g., optical flow) or in point clouds.

3.4 Lifelong SLAM

Lázaro et al. [31] implemented map updates for ... Mielle et al. [32] ...

3.5 Map Point Observability

Derner et al. [33] ... Berrio et al. [34, 35] ... Alcantarilla et al. [36] ...

4 Implementation

4.1 Definitions and Functions

Future sections explore the mathematical basis and implementation of the proposed system. To provide a common reference, useful definitions and constructs are defined here.

Definitions

$d_{\min}$ and $d_{\max}$: The minimum and maximum distances at which a map point may be considered visible to the camera.

v: A viewpoint in the domain $v \in \mathbb{S}^2 \times [d_{\min}, d_{\max}] \to \mathbb{R}^3$.

o: An observation of a map point defined as $o = (v, s)$, where v is the viewpoint and $s \in \{0.0, 1.0\}$ indicates whether the point was seen (1.0) or not seen (0.0) from v.

$\boldsymbol{O}^n$: A vector of n observations in the form $\{o_0, \ldots, o_n\}$.

$P(E)$: The probability that the map point exists.

$P(S, v)$: The probability that the map point is visible from a given viewpoint v.

ε_{neg}: The probability of a false negative (not observing a map point when it is actually observable).

ε_{pos}: The probability of a false positive (observing a map point when it is actually unobservable).

α: The damping coefficient applied to updates of $P(E)$.

ρ_t: The estimated overall observability of the map point at time t.

η: The damping coefficient applied to updates of ρ.

β: The feedback propagated from the Bayesian update step back to the KNN model, calculated as $\hat{P}(E)_t - \hat{P}(E)_{t-1}$.

KNN: An instance of the KNN observability model.

EPE: An instance of the existence probability estimator.

VBEE: An instance of the viewpoint-based existence estimator, which contains both an existence probability estimator and a KNN model.

$\boldsymbol{\theta}_{\text{KNN}}$: A vector of parameters specific to the KNN model in the form $\{\theta_{\text{KNN},1}, \ldots, \theta_{\text{KNN},n}\}$, where n is the number of parameters.

$\boldsymbol{\theta}_{\text{EPE}}$: A vector of parameters for the existence probability estimator in the form $\{\varepsilon_{\text{neg}}, \varepsilon_{\text{pos}}\}$.

$\boldsymbol{\theta}_{\text{VBEE}}$: A vector of parameters for the viewpoint-based existence estimator in the form $\{\text{KNN}, \hat{P}(E)_0, \alpha, \rho_0, \eta\}$.

Functions

VBEE. Initialize($\boldsymbol{\theta}_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}_{\text{EPE}}$): Initializes the viewpoint-based existence estimator.

VBEE. Update(o): Updates the estimator with a new observation o, returning the updated existence probability $\hat{P}(E)$.

VBEE. Query(): Queries the estimator for the current estimate of the map point's existence probability $\hat{P}(E)$.

KNN. Initialize($\boldsymbol{\theta}_{\text{KNN}}$): Initializes the KNN model with parameters $\boldsymbol{\theta}_{\text{KNN}}$.

KNN. Estimate(v): Returns the KNN model's estimated probability of seeing the map point from viewpoint v, given the point exists: Estimate(v) $= \hat{P}(S, v|E)$.

KNN. Integrate(o, β): Integrates a new observation o into the KNN model's internal representation, using feedback β to adjust for the direction and magnitude of the change in existence probability.

EPE. Initialize($\boldsymbol{\theta}_{\text{EPE}}$): Initializes the existence probability estimator with parameters $\boldsymbol{\theta}_{\text{EPE}}$.

EPE. Update($o, \hat{P}(E)_t, \hat{P}(S, v|E)$): Updates the map point's existence probability $\hat{P}(E)_{t+1}$ based on o, the prior estimate $\hat{P}(E)_t$, and the current KNN estimate $\hat{P}(S, v|E)$.

4.2 Method Overview

The method described here iteratively estimates and refines the probability of a map point's existence through the integration of new observations and historical observability priors. This is accomplished through the use of a model for estimating the observability of map points from historical observations, and a Bayesian updater which uses this model to update the existence probability of the map point based on new observations. This chapter gives an overview of the full existence estimation framework, followed by a detailed description of the observability modeling approach, the Bayesian existence probability estimator, and the impact of overall observability on the existence estimation process.

4.2.1 The Existence Estimation Framework

The existence estimation framework combines an observability model with a Bayesian existence probability estimator. A high-level view is shown in Figure 7.

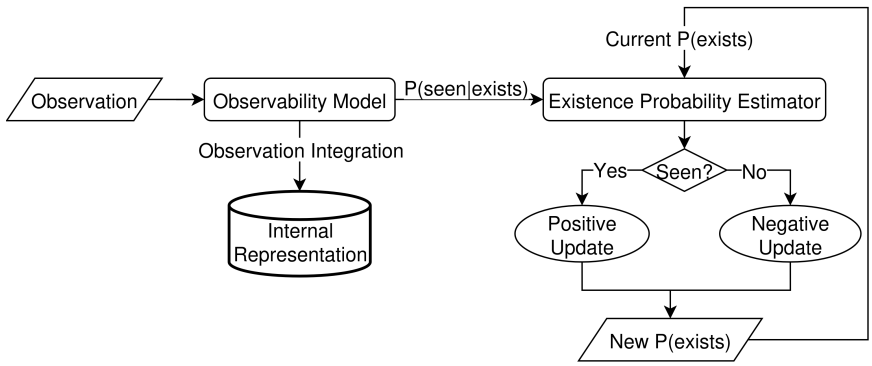

Figure 7: A flowchart representing the flow of data through the proposed existence estimation framework.

The observability model estimates $P(S, v|E)$, the probability of observing a map point from viewpoint v given that the point exists, using historical observations. This estimate is

then used by the existence probability estimator to update the existence probability $P(E)$ of the map point in a Bayesian manner. After the update, the new observation o is integrated into the model's internal representation, with feedback β from the existence probability update used to adjust the strength of integration.

4.2.2 The Observability Model

Map points are conceptualized as static, infinitesimally small points in 3D Euclidean space. Let the observability of a map point p from a viewpoint $v = \{\theta, \phi, d\} \in [0, 2\pi) \times \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \times [0, d_{\max})$ be represented by

$$\text{observable}_p(v) : V \to \{0, 1\},$$

which returns 1 if p is observed from v and 0 otherwise. An observation of point p is then defined as $o_p = (v, s)$, where $s \in \{0, 1\}$ denotes whether the point was seen. A set of n observations is $\boldsymbol{O}_p^n = \{o_{p0}, \ldots, o_{pn}\}$.

For illustrative purposes, later figures may use a simplified 2D representation $v^{2D} = \{\theta, d\}$. Figure 8 shows an example of such a plot.

Constructing the exact global observability surface would require infinite observations across the full domain of viewpoints, which is infeasible. To satisfy the constant-size requirement in section 1.3, the observability model maintains a finite, bounded representation.

The model serves two roles: 1. Estimating $P(S, v|E)$ from a given viewpoint. 2. Integrating new observations into its internal representation.

The objective is to minimize the total error between the model estimate and the true observability:

$$\int \left| \hat{P}(S, v|E) - \text{observable}_p(v) \right| dV.$$

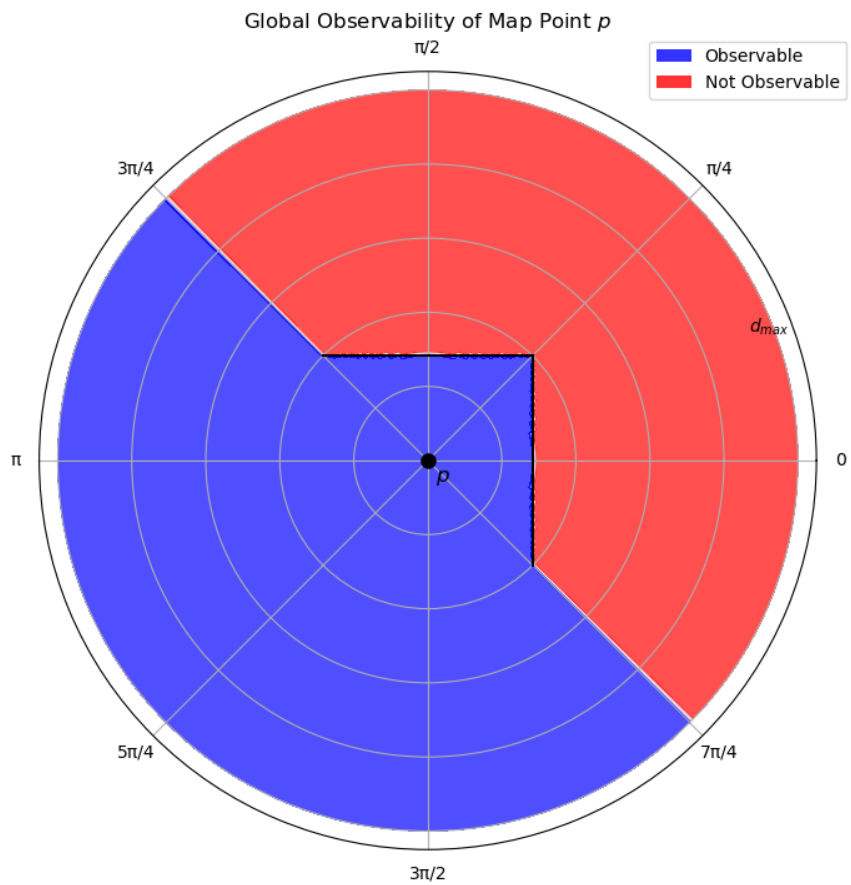

Figure 8: A 2D representation of the global observability of a map point p near two orthogonal obstructions.

4.2.3 Existence Probability Estimator

The probability of existence $P(E)$ is updated with each new observation using Bayesian inference. Let $P(E)_t$ denote the prior probability of existence before the observation, and $P(E)_{t+1}$ the posterior after incorporating it.

$$P(E)_{t+1} = \begin{cases} \dfrac{P(S, v|E)\, P(E)_t}{P(S, v)} & \text{if } s = 1, \\ \dfrac{P(\neg S, v|E)\, P(E)_t}{P(\neg S, v)} & \text{if } s = 0. \end{cases}$$

The marginal likelihood is

$$P(S, v) = P(S, v|E)\, P(E)_t + P(S, v|\neg E)\, (1 - P(E)_t).$$

False matches are possible when the map point does not exist. Let

$$\varepsilon_{\text{pos}} = P(S, v|\neg E) \approx 0,$$

representing the false positive rate. Assuming the model provides an estimate $\hat{P}(S, v|E) \equiv m(v)$, the update simplifies to

$$P(E)_{t+1} = \begin{cases} \dfrac{m(v)\, P(E)_t}{m(v)\, P(E)_t + \varepsilon_{\text{pos}}(1 - P(E)_t)} & \text{if } s = 1, \\ \dfrac{(1 - m(v))\, P(E)_t}{(1 - m(v))\, P(E)_t + (1 - \varepsilon_{\text{pos}})(1 - P(E)_t)} & \text{if } s = 0. \end{cases}$$

4.2.4 Overall Observability

The update above is overly sensitive in cases where a map point has historically been highly observable. For instance, if $m(v) = 1$, then a single negative observation ($s = 0$) would drive $P(E)_{t+1}$ close to zero. To reduce this sensitivity, a damping coefficient $\alpha \in [0, 1]$ is applied:

$$P(E)_{t+1} = \begin{cases} \alpha \cdot \dfrac{m(v)\, P(E)_t}{m(v)\, P(E)_t + \varepsilon_{\text{pos}}(1 - P(E)_t)} & \text{if } s = 1, \\ \alpha \cdot \dfrac{(1 - m(v))\, P(E)_t}{(1 - m(v))\, P(E)_t + (1 - \varepsilon_{\text{pos}})(1 - P(E)_t)} & \text{if } s = 0. \end{cases}$$

To further improve robustness, the model incorporates an overall observability parameter ρ, defined as the proportion of viewpoints from which the point has historically been observable. This is adaptively updated using

$$\rho \;\leftarrow\; \rho + \eta \cdot \big(s - m(v)\big),$$

where η controls the adaptation rate. In effect, ρ moderates the influence of each observation, ensuring that negative evidence in highly observable regions has more weight than

in rarely observable ones.

4.3 System Implementation

This section describes the implementation of the existence estimation framework described in section 4.2. Implementation details of the KNN observability model, the Bayesian updater, and the Viewpoint-Based Existence Estimator library are provided. Pseudocode implementations of key functions are included to illustrate the implementation details of the framework.

4.3.1 Existence Estimation Framework Implementation

The viewpoint-based existence estimation framework is implemented as a C++ library called `libvbee`. This library facilitates the integration of VBEE into KV-SLAM systems by providing a standardized interface for observability estimation and existence probability estimation. The library provides the VBEE class, which configures the KNN model and provides functions for updating and querying the existence probability of a map point. Pseudocode for the initialization, update, and query functions is provided below.

The initialization function in Algorithm 1 sets up the VBEE instance with the specified parameters, initializing the KNN model and the existence probability estimator.

Algorithm 1 VBEE Initialization: $\text{VBEE.Initialize}(\boldsymbol{\theta}_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}_{\text{EPE}})$

Require: $\boldsymbol{\theta}_{\text{VBEE}}$: VBEE Parameters
Require: $\boldsymbol{\theta}_{\text{KNN}}$: KNN Parameters
Require: $\boldsymbol{\theta}_{\text{EPE}}$: Existence Probability Estimator Parameters
  global $p_e, \alpha, \rho, \eta \leftarrow \boldsymbol{\theta}_{\text{VBEE}}$
  global $knn \leftarrow \text{KNN.Initialize}(\boldsymbol{\theta}_{\text{KNN}})$
  global $epe \leftarrow \text{EPE.Initialize}(\boldsymbol{\theta}_{\text{EPE}})$

The update function in Algorithm 2 integrates a new observation, determining an updated existence probability based on the observation and the KNN estimate. The KNN model is

updated with feedback, and the overall observability is updated based on the difference between the observation's state and the KNN estimate.

Algorithm 2 VBEE Update: VBEE. Update(o)

Require: o: New Observation
Ensure: $\hat{P}(E)$: Updated Existence Probability

```
prior ← p_e
model_estimate ← knn. Estimate(o.v)
posterior ← epe. Update(o, prior, model_estimate)
weight ← (1 − ρ) · α
p_e ← prior · (1 − weight) + posterior · weight
β ← p_e − prior
knn. Update(o, β)
ρ ← ρ + η · (o.s − model_estimate) return p_e
```

The query function simply returns the current existence probability $P(E)$.

4.3.2 KNN Observability Model

This thesis uses a K-nearest neighbors (KNN) method for modeling map point observability. The KNN model estimates $P(S, v|E)$ as the ratio of positive observations within the k nearest neighbors of the query viewpoint. Since infinite observations cannot be stored, the model maintains a finite buffer of n observations. The angular distance between viewpoints is also considered, as distant observations may be irrelevant. Finally, the model incorporates feedback from the existence probability estimator to decide whether or not to integrate a new observation.

The parameter vector is:

$$\boldsymbol{\theta}_{\text{KNN}} = \{k, n, \tau_{\text{angle}}, \tau_\beta\}$$

where k is the number of neighbors to consider, n is the maximum number of stored observations, τ_{angle} is the maximum angular distance for neighbors, and τ_β is the minimum feedback magnitude required for integration.

The estimation function is shown in Algorithm 3, returning the ratio of positive neighbors,

or 0.5 if no neighbors are found.

Algorithm 3 KNN Estimate(v)

Require: $\boldsymbol{O}_{prev}$: Stored observations
Require: k, τ_{angle}: KNN parameters
Ensure: $\hat{P}(S, v|E)$: Estimated probability of observation

```
function Estimate()(v)
    candidates ← [ ]
    for all o ∈ O_prev do
        if Δθ(o.v, v) < τ_angle then
            add o to candidates
        end if
    end for
    sort candidates by distance to v
    neighbors ← first min(k, length(candidates))
    if neighbors is empty then
        return 0.5
    end if
    seen_count ← sum of o.s in neighbors
    return seen_count/length(neighbors)
end function
```

The update function (Algorithm 4) integrates new observations. If the buffer is not full, the observation is added. If the feedback magnitude is below τ_β, the observation is ignored. Otherwise, the observation replaces the nearest inconsistent stored observation.

Algorithm 4 KNN Update(o, β)

Require: $\boldsymbol{O}_{prev}$: Stored observations
Require: n, τ_β: Parameters
  function Update()(o, β)
    if `length`($\boldsymbol{O}_{prev}$) $< n$ **then**
      add o
      return
    end if
    if $|\beta| < \tau_\beta$ **then**
      return
    end if
    if $o.s = 1$ **then**
      replace nearest not-seen observation with o
    else
      replace nearest seen observation with o
    end if
  end function

The accuracy and performance of the KNN model will be evaluated in section 5.

4.3.3 Existence Probability Estimator

The Existence Probability Estimator (EPE) updates the probability $P(E)$ of a map point's existence. It is initialized with $\boldsymbol{\theta}_{\text{EPE}} = \{\varepsilon_{\text{neg}}, \varepsilon_{\text{pos}}\}$. The update function takes an observation, the prior $P(E)_t$, and the KNN estimate $\hat{P}(S, v|E)$ as inputs, and computes the posterior $P(E)_{t+1}$ with a Bayesian update. Algorithm 5 shows the implementation.

Algorithm 5 Bayesian Update Step

Require: Observation $o = (v, s)$
Require: Prior $\hat{P}(E)_t \in [0, 1]$
Require: KNN estimate $\hat{P}(S, v|E)$
  if $o.s = 1$ **then**
    $posterior \leftarrow \frac{\hat{P}(S,v|E)(1-\varepsilon_{\text{neg}})\cdot\hat{P}(E)_t}{\hat{P}(S,v|E)(1-\varepsilon_{\text{neg}})\cdot\hat{P}(E)_t+\varepsilon_{\text{pos}}(1-\hat{P}(E)_t)}$
  else
    $posterior \leftarrow \frac{(1-\hat{P}(S,v|E)(1-\varepsilon_{\text{neg}}))\cdot\hat{P}(E)_t}{(1-\hat{P}(S,v|E)(1-\varepsilon_{\text{neg}}))\cdot\hat{P}(E)_t+(1-\varepsilon_{\text{pos}})(1-\hat{P}(E)_t)}$
  end if
  return $posterior$

4.4 ORB-SLAM3 Integration

The VBEE library is designed to be easily integrated into existing KV-SLAM systems. This section covers the integration of libVBEE into ORB-SLAM3. As stated in section 2.2, ORB-SLAM3 was selected due to its wide adoption, high performance, and inclusion of several SLAM extensions which can benefit from outdated map point culling. Section 5 will quantify the effects of the VBEE library integration into ORB-SLAM3 on several performance indicators.

Integration begins in the `MapPoint` class, where each map point is given an instance of the VBEE class. This allows each map point to have its viewpoint-based existence probability managed separately. Each VBEE instance is initialized with the same parameter vectors $\boldsymbol{\theta}_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}_{\text{EPE}}$ in the `MapPoint` constructor, ensuring that all map points share the same configuration. The VBEE instance manages the existence probability of the map point throughout its lifetime.

To update the existence probability, the map point must receive positive and negative observations. Positive observations are straightforward to retrieve: after a new frame has been processed, all the observed map points are available in the `Frame` class. Negative observations require a more complex approach, as they are not directly available. The function FindMapPointsInCameraView, shown in Algorithm 6, is implemented in the `Map` class to retrieve all map points that are within the camera's view. This function uses the current camera pose, the camera's projection matrix, and the active `Map` instance to find all map points that would be projected into the camera's view. The function returns a vector of map points in view. Negative observations are then determined by checking whether a map point is in the camera's view but not in the frame's observed map points.

Algorithm 6 FindMapPointsInCameraView

```
function FindMapPointsInCameraView(camera_pose, projection_matrix, map)
    camera_view_points ← []
    for map_point in map.GetAllMapPoints() do
        if IsPointInCameraView(map_point, camera_pose, projection_matrix) then
            if map_point not in observed_map_points then
                camera_view_points.append(map_point)
            end if
        end if
    end for
    return camera_view_points
end function
```

The main loop of ORB-SLAM3's Tracking thread is modified to update all map points in the camera's view after each frame is processed. The function UpdateExistenceProbabilities, shown in Algorithm 7, updates the existence probabilities of all map points in the camera's view. This function retrieves the camera's pose and projection matrix, finds all map points in the camera's view, and updates their existence probabilities based on positive and negative observations.

Algorithm 7 UpdateExistenceProbabilities

```
Require: frame: Current Frame
function UpdateExistenceProbabilities
    all_map_points ← FindMapPointsInCameraView()
    seen_points ← all map points observed in frame
    unseen_points ← all points in all_map_points that are not in seen_points
    for map_point in seen_points do
        observation ← positive observation of map_point from camera_pose
        map_point.VBEE.Update(observation)
    end for
    for map_point in unseen_points do
        observation ← negative observation of map_point from camera_pose
        map_point.VBEE.Update(observation)
    end for
end function
```

C++ implementations of these functions are provided in libVBEE for ease of integration, and can be seen in section 6.2. With these modifications, each map point now has an existence

probability estimate that is updated based on the observations received during the SLAM process. However, these probabilities are not yet used to cull outdated map points. The next section explores how culling logic is implemented into ORB-SLAM3 to make use of the existence probabilities provided by libVBEE.

4.4.1 Culling Logic Implementation

ORB-SLAM3 performs map point culling as part of the `LocalMapping` thread. The process looks through the list of recently added map points, and checks them against several criteria.

- Another thread has marked the map point as "bad"
- The "found ratio" drops below 0.25
- The map point is seen too few times in the first 3 keyframes

These criteria are effective for quickly removing false matches, but as the map point culling process only operates only recently added map points, it has no effect on map points which are considered established which may no longer exist.

4.5 Dataset Creation

Two datasets were created to test the isolated performance of the VBEE library, and the integrated performance of VBEE with ORB-SLAM3 respectively The first dataset referred to as the Observability Scenarios Dataset, consists of a set of points, each with their own unique observability properties. These points can be queried to determine if a point can be seen from a given viewpoint, allowing the full VBEE pipeline to be exercised without a SLAM implementation. The second dataset known as the Episodic Visual SLAM Dataset is a stereo visual SLAM dataset in the style of the EuRoC MAV SLAM dataset, but with extensions for episodic operation. This section details the construction and intention behind these datasets.

4.6 Observability Scenarios Dataset

The observability scenarios dataset is intended to represent the domain of observability scenarios which may apply to map points seen during SLAM operations. A map point's observability can be represented as the union of three sets of viewpoints: Viewpoints from which the map point is visible, viewpoints from which the map point is not visible (obstructed), and viewpoints which are inaccessible to the observer (e.g. under a floor or inside a wall). These three sets cover the domain of viewpoints for a map point and represent the unique observability scenario for that map point.

For the creation of the observability scenarios dataset, rather than classifying specific viewpoints into specific sets, the proportional sizes of these sets are defined, and a generative process is used to create a scenario matching these proportional sizes. The process of generating an observability scenario for a desired parameterization is shown in Algorithm 8. This algorithm randomly generates inaccessible and obstructed areas around a central point, then samples random viewpoints to estimate the relative proportions of each set. If the proportional set sizes match the desired proportions within a defined tolerance, the observability scenario is returned. Otherwise, the number of inaccessible and obstructed areas are adjusted and regenerated until a valid solution is found.

Algorithm 8 GenerateObservabilityScenario

```
function GenerateObservabilityScenario
end function
```

The final dataset consists of observability scenarios generated for all pairings of 5% inaccessible to 95% inaccessible and 5% obstructed to 95% obstructed in 5% increments, giving a total of 361 unique observability scenarios. A subset of cutaways from the dataset is shown in Figure 9.

Figure 9: 2D cutaways of examples from the Observability Scenarios Dataset

4.6.1 Episodic Visual SLAM Dataset

To evaluate VBEE performance in a real-world SLAM system, a stereo visual dataset was created consisting of a series of trajectories through a household environment. No dynamic changes occur during the recording of a single trajectory, but environmental changes are introduced between trajectories, matching the episodic run modality described in section 2. Figure 10 shows example images from each trajectory at roughly the same location.

Figure 10: Example images from each trajectory of the episodic visual SLAM dataset

This dataset is intended to replicate realistic episodic SLAM operation in a household environment. Ground truth is difficult to obtain in large-scale 3D environments, so evaluation focuses on the SLAM system's ability to maintain a consistent map across multiple trajectories. LibVBEE is integrated into ORB-SLAM3, and performance is evaluated by measuring its ability to identify and remove outdated map points while preserving map consistency across episodes.

4.7 Parameter Optimization

Before experimentation can take place on the SLAM system, the parameters of libVBEE must be tuned to optimize performance. Parameters are split between the VBEE framework and the observability model. The VBEE and model parameters are tightly coupled through feedback, making optimization difficult as the parameters of one system affect the other. As a solution, parameter optimization is performed in three phases. The first phase freezes the VBEE parameters to reasonable values while optimizing the observability model parameters. The second phase freezes the observability model parameters to the values obtained in Phase 1 while optimizing the VBEE parameters. The final phase fine-tunes both parameter sets simultaneously, ideally achieving stable performance for the system.

Phase 1 - Observability Model Tuning

The observability model has parameters $\boldsymbol{\theta}_{\text{KNN}} = \{k, n, \tau_{\text{angle}}, \tau_{\beta}\}$, which are tuned to minimize the following objectives:

- n_{converge}: The number of observations necessary for the VBEE system to converge to a stable estimate of $P(E)$
- $\epsilon_{\text{converge}}$: The observability model's error after n_{converge} observations
- n_{adapt}: The number of observations necessary for the system to re-converge to a stable estimate of $P(E)$ after changing the point's observability
- ϵ_{adapt}: The observability model's error after n_{adapt} observations
- t_{update}: The average time in ms needed to perform one estimation and one update
- size: The total storage in bytes needed to serialize one map point's observability

In this first phase, the VBEE and EPE parameters are locked to the following values:

$$\boldsymbol{\theta}^*_{\text{EPE}} = \{\varepsilon_{\text{neg}} = 0.1, \varepsilon_{\text{pos}} = 0.001\}$$
$$\boldsymbol{\theta}^*_{\text{VBEE}} = \{\hat{P}(E)_0 = 0.9, \alpha = 0.1, \rho_0 = 0.5, \eta = 0.5\}$$

The observability model parameters $\boldsymbol{\theta}_{\text{KNN}}$ are optimized to minimize the objectives using the objective functions described below.

The function ObjectiveNConverge shown in Algorithm 9 determines the average number of observations needed for the system to converge to a stable $\hat{P}(E)$ for each observability scenario in the dataset. The system is considered converged when the standard deviation over the last $w = 100$ samples drops below $\sigma_{\text{converge}} = 0.2$. If the system does not converge within $m_{\max} = 1000$ observations, the test is considered failed, and the optimization is penalized accordingly.

Algorithm 9 ObjectiveNConverge

Require: *vbee*: VBEE instance initialized with $\boldsymbol{\theta}^*_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}^*_{\text{EPE}}$
Require: *observability_scenarios*: Observability Scenario Dataset

```
function ObjectiveNConverge
    all_n_convergences ← [ ]
    for os in observability_scenarios do
        reset vbee
        window ← Queue()
        n_converge ← −1
        for i = 0 to m_max do
            v ← random viewpoint
            new_P̂(E) ← VBEE.Update(o = (v, os.Query()(v)))
            push new_P̂(E) onto window
            if window size > w then
                pop window
                if std dev(window) < σ_converge then
                    n_converge ← i
                    break
                end if
            end if
        end for
        if n_converge = −1 then
            append 2 * m_max to all_n_convergences
        else
            append i to all_n_convergences
        end if
    end for
    return average all_n_convergences
end function
```

The function ObjectiveEConverge shown in Algorithm 10 determines the average error between the observability model and the observability scenario after the system has converged across all observability scenarios. Model error is calculated as the sum of differences between VBEE.Query() and the true observability from the dataset over $j = 1000$ random viewpoints. Once again, if the system does not converge to a stable probability estimate, the system is penalized.

Algorithm 10 ObjectiveEConverge

```
Require: vbee: VBEE instance initialized with θ*_VBEE, θ_KNN, θ*_EPE
Require: observability_scenarios: Observability Scenario Dataset
  function ObjectiveEConverge
      all_e_convergences ← [ ]
      for os in observability_scenarios do
          reset vbee
          error ← 0
          window ← Queue()
          for m_max times do
              [Same as Algorithm 9, lines 8-13]
              if window size > w then
                  pop window
                  if std dev(window) < σ_converge then
                      for j times do
                          v ← random viewpoint
                          true_obs ← os.Query()(v)
                          model_obs ← vbee.model.Estimate(v)
                          error ← error + |true_obs − model_obs|
                      end for
                      break
                  end if
              end if
          end for
          if error = 0 then
              append 1000 to all_e_convergences
          else
              append error to all_e_convergences
          end if
      end for
      return average all_e_convergences
  end function
```

The functions ObjectiveNAdapt and ObjectiveEAdapt are implemented similarly but perform two convergence loops per scenario. Between the loops, the active scenario is replaced with a predetermined partner scenario selected to be significantly different from the original but with some observability overlap.

The function ObjectiveTUpdate, shown in Algorithm 11, determines the average time to perform VBEE. Update() for 1000 random observations, averaged for every scenario.

Algorithm 11 ObjectiveTUpdate

Require: *vbee*: VBEE instance initialized with $\boldsymbol{\theta}^*_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}^*_{\text{EPE}}$
Require: *observability_scenarios*: Observability Scenario Dataset

```
function ObjectiveTUpdate
    all_t_updates ← [ ]
    for os in observability_scenarios do
        reset vbee
        avg_t_update ← 0
        for 1000 times do
            v ← random viewpoint
            t_0 ← current time in ms
            new_P̂(E) ← VBEE.Update(o = (v, os.Query()(v)))
            t_1 ← current time in ms
            avg_t_update ← avg_t_update + (t_1 − t_0)
        end for
        append (avg_t_update/1000) to all_t_updates
    end for
    return average all_t_updates
end function
```

The function ObjectiveSize calculates the total size in bytes necessary to store all information required to reconstruct a serialized map point VBEE system in the same state as when it was serialized. This reconstruction requires:

- All parameter vectors $\boldsymbol{\theta}_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}_{\text{EPE}}$
- The current estimated existence probability $\hat{P}(E)$
- The current observability ρ
- The list of observations from the observability model

The only item in this list with a size dependent on the parameter vectors is the observations list, which has a maximum capacity of n, provided in $\boldsymbol{\theta}_{\text{KNN}}$. All other sizes are not dependent on the input parameters. The serialization of an observation requires three floating-point values for the x, y, and z coordinates in the map point's frame, and a boolean value for whether the observation is positive or negative. If a float is assumed to be four bytes in size and the boolean one byte, then ObjectiveSize can be implemented to simply return $5 * n$.

The final multi-objective function is shown in Algorithm 12. This function returns an

objective vector of six values, each related to one of the performance criteria defined previously. At this stage, the system is able to undergo multi-objective optimization to find values that minimize each of the six objectives.

Algorithm 12 Observability Model Objective Function

Require: $\boldsymbol{\theta}^*_{\text{VBEE}}, \boldsymbol{\theta}^*_{\text{EPE}}$: Static VBEE and EPE parameters
  function ObjectiveObservabilityModel($\boldsymbol{\theta}_{\text{KNN}}$)
    $vbee \leftarrow$ VBEE.Initialize($\boldsymbol{\theta}^*_{\text{VBEE}}, \boldsymbol{\theta}_{\text{KNN}}, \boldsymbol{\theta}^*_{\text{EPE}}$)
    $n_{converge} \leftarrow$ ObjectiveNConverge()
    `reset` $vbee$
    $\epsilon_{converge} \leftarrow$ ObjectiveEConverge()
    `reset` $vbee$
    $n_{adapt} \leftarrow$ ObjectiveNAdapt()
    `reset` $vbee$
    $\epsilon_{adapt} \leftarrow$ ObjectiveEAdapt()
    `reset` $vbee$
    $t_{update} \leftarrow$ ObjectiveTUpdate()
    `reset` $vbee$
    $size \leftarrow$ ObjectiveSize()
    return $(n_{converge}, \epsilon_{converge}, n_{adapt}, \epsilon_{adapt}, t_{update}, size)$
  end function

The Python library Optuna [37] was selected to perform the multi-objective optimization of the system's parameters. Optuna is a hyperparameter optimization framework that allows for the discovery of Pareto optimal [38] parameter sets.

Phase 2 - VBEE and EPE Tuning

In the second phase, the observability model parameters are frozen to the values obtained in Phase 1, and the VBEE and EPE parameters are tuned. The VBEE parameters $\boldsymbol{\theta}_{\text{VBEE}} = \{\hat{P}(E)_0, \alpha, \rho_0, \eta\}$ are tuned to minimize the same objectives as in Phase 1, while the EPE parameters $\boldsymbol{\theta}_{\text{EPE}} = \{\varepsilon_{\text{neg}}, \varepsilon_{\text{pos}}\}$ are tuned to minimize the following objectives:

4.8 Experimental Setup

This section describes the experiments conducted to evaluate the performance of the observability model and to determine the impact of libVBEE on SLAM systems. These experiments are defined here, and the results are presented in section 5.

5 Results and Analysis

5.1 EPE Analysis and Optimization

5.2 Evaluation Metrics

5.3 SLAM System Configurations

5.4 Parameter Optimization

5.4.1 Existence Estimation Framework Parameter Selection

There are various tunable parameters for the observability models and the Bayesian update. These parameters have significant impact on the accuracy and performance of the existence probability estimation, so a systematic process is outlined here to select ideal parameterizations. The update step and the model output are dependent on one-another, so a multi-stage approach is used to select initial parameters, followed by a joint optimization.

An initial parameterization for the Bayesian update step must be selected for the optimization of model parameters, which is accomplished in the first stage. The second stage exclusively optimizes the parameters of the observability models, and the final stage performs joint optimization on both the observability models and the Bayesian update.

Stage 1 - Bayesian Update Initial Parameter Selection

Parameters of Interest

- Initial pExists
- Initial Observability
- Observability Update factor
- Damping Coefficient
- False Positive Rate

- False Negative Rate

The Bayesian update should cause p_{exists} to accurately track the true existence of the point, even when the point is only observable from a small subset of viewpoints. Additionally, the damping factor should prevent p_{exists} from dropping rapidly after a missed observation to prevent the point from being eliminated due to a temporary occlusion.

To set these initial values rigorously, an experiment is set up to provide the Bayesian updater with random observations of a point with a set visibility, with the goal of minimizing the error between p_{exists} and the true existence of the point.

The model is set up to always report $p_{seen} = 0.5$. Observability is locked to the true observability of the point. Initial values of false positive and negative rates are selected based on the assumption that a false negative (the camera failing to observe a map point despite its visibility) is significantly higher than that of a false positive (the camera claiming to observe a point which is not visible). These parameters are highly dependent on the SLAM implementation, and will therefore be fine-tuned after integration with ORB-SLAM3, but for early stage are given settings of $\varepsilon_{\text{neg}} = 0.05$ and $\varepsilon_{\text{pos}} = 0.001$.

The observability of the point will drop from 0.8 to 0.2 in increments of 0.1 every 100 observations. Following this, the observability will rise back to 0.9 at the same rate. The observation probability then drops to 0.5, then 0.2, and back. Finally, the observability is set to 0. This experiment verifies the following:

1. The probability of existence never drops below 0.2 (until the point is removed)

2. The error between the estimated and true probability is minimized

3. The probability of existence is prevented from changing so quickly as to prematurely delete the point

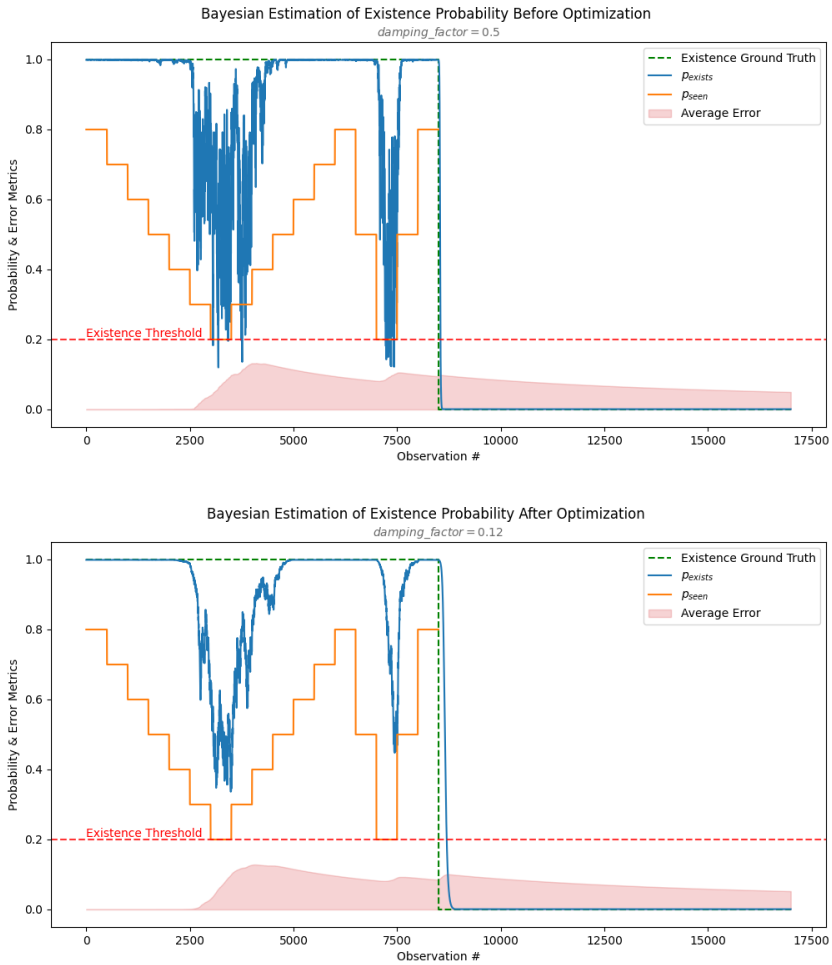

Figure 11: Damping Coefficient Selection Results.

5.5 Results

5.5.1 Quantitative Evaluation

5.5.2 Qualitative Evaluation

5.5.3 Ablation Study

6 Conclusion and Future Work

6.1 Future Work

References

[1] Trey Smith et al. "ASTROBEE: A NEW PLATFORM FOR FREE-FLYING ROBOTICS ON THE INTERNATIONAL SPACE STATION". In: ().

[2] Ryan Soussan et al. "AstroLoc: An Efficient and Robust Localizer for a Free-flying Robot". In: *2022 International Conference on Robotics and Automation (ICRA)*. 2022 International Conference on Robotics and Automation (ICRA). May 2022, pp. 4106–4112. DOI: 10.1109/ICRA46639.2022.9811919.

[3] Roberto Carlino et al. "Lessons Learned from Astrobee Operations on the International Space Station".

[4] Tom Zurales et al. *Collaborative Sensing and Mapping for IVA Robots Phase II Final Report*. Apr. 5, 2024.

[5] Paul Robinette, Ayanna M. Howard, and Alan R. Wagner. "Effect of Robot Performance on Human–Robot Trust in Time-Critical Situations". In: *IEEE Transactions on Human-Machine Systems* 47.4 (Aug. 2017), pp. 425–436. ISSN: 2168-2305. DOI: 10.1109/THMS.2017.2648849.

[6] Saeid Nahavandi et al. "A Comprehensive Review on Autonomous Navigation". In: *ACM Computing Surveys* 57.9 (Sept. 30, 2025), pp. 1–67. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3727642.

[7] Randall Smith, Matthew Self, and Peter Cheeseman. "Estimating Uncertain Spatial Relationships in Robotics*". In: *Machine Intelligence and Pattern Recognition*. Ed. by John F. Lemmer and Laveen N. Kanal. Vol. 5. Uncertainty in Artificial Intelligence. North-Holland, Jan. 1, 1988, pp. 435–461. DOI: 10.1016/B978-0-444-70396-5.50042-X.

[8] Stephen Se, David Lowe, and J.J. Little. "Mobile Robot Localization and Mapping with Uncertainty Using Scale-Invariant Visual Landmarks". In: *The International Journal of Robotics Research* 21 (Aug. 1, 2002), pp. 735–760. DOI: 10.1177/027836402761412467.

[9] Davison. "Real-Time Simultaneous Localisation and Mapping with a Single Camera". In: *Proceedings Ninth IEEE International Conference on Computer Vision*. Ninth IEEE International Conference on Computer Vision. Oct. 2003, 1403–1410 vol.2. DOI: 10.1109/ICCV.2003.1238654.

[10] D.G. Lowe. "Object Recognition from Local Scale-Invariant Features". In: *Proceedings of the Seventh IEEE International Conference on Computer Vision*. The Seventh IEEE International Conference on Computer Vision. Vol. 2. Sept. 1999, 1150–1157 vol.2. DOI: 10.1109/ICCV.1999.790410.

[11] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. "SURF: Speeded Up Robust Features". In: *Computer Vision – ECCV 2006*. Ed. by Aleš Leonardis, Horst Bischof, and Axel Pinz. Berlin, Heidelberg: Springer, 2006, pp. 404–417. ISBN: 978-3-540-33833-8. DOI: 10.1007/11744023_32.

[12] Ethan Rublee et al. "ORB: An Efficient Alternative to SIFT or SURF". In: *2011 International Conference on Computer Vision*. 2011 International Conference on Computer Vision. Nov. 2011, pp. 2564–2571. DOI: 10.1109/ICCV.2011.6126544.

[13] Michael Burri et al. "The EuRoC Micro Aerial Vehicle Datasets". In: *The International Journal of Robotics Research* 35 (Jan. 25, 2016). DOI: 10.1177/0278364915620033.

[14] Martin A. Fischler and Robert C. Bolles. "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography". In: *Commun. ACM* 24.6 (June 1, 1981), pp. 381–395. ISSN: 0001-0782. DOI: 10.1145/358669.358692.

[15] H. C. Longuet-Higgins. "A Computer Algorithm for Reconstructing a Scene from Two Projections". In: *Nature* 293.5828 (Sept. 1981), pp. 133–135. ISSN: 1476-4687. DOI: 10.1038/293133a0.

[16] R.I. Hartley. "In Defense of the Eight-Point Algorithm". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 19.6 (June 1997), pp. 580–593. ISSN: 1939-3539. DOI: 10.1109/34.601246.

[17] Mark Cummins and Paul Newman. "Probabilistic Appearance Based Navigation and Loop Closing". In: *Proceedings 2007 IEEE International Conference on Robotics and Automation*. 2007 IEEE International Conference on Robotics and Automation. Apr. 2007, pp. 2042–2048. DOI: 10.1109/ROBOT.2007.363622.

[18] Dorian Galvez-López and Juan D. Tardos. "Bags of Binary Words for Fast Place Recognition in Image Sequences". In: *IEEE Transactions on Robotics* 28.5 (Oct. 2012), pp. 1188–1197. ISSN: 1941-0468. DOI: 10.1109/TRO.2012.2197158.

[19] Xuesong Shi et al. *Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM*. Mar. 14, 2020. DOI: 10.48550/arXiv.1911.05603. arXiv: 1911.05603 [cs]. URL: http://arxiv.org/abs/1911.05603 (visited on 03/11/2025). Pre-published.

[20] Carlos Campos et al. "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM". In: *IEEE Transactions on Robotics* 37.6 (Dec. 2021), pp. 1874–1890. ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TRO.2021.3075644. arXiv: 2007.11898 [cs].

[21] Zetao Chen et al. "Deep Learning Features at Scale for Visual Place Recognition". In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 2017 IEEE International Conference on Robotics and Automation (ICRA). May 2017, pp. 3223–3230. DOI: 10.1109/ICRA.2017.7989366.

[22] Tayyab Naseer et al. "Vision-Based Markov Localization across Large Perceptual Changes". In: *2015 European Conference on Mobile Robots (ECMR)*. 2015 European Conference on Mobile Robots (ECMR). Sept. 2015, pp. 1–6. DOI: 10.1109/ECMR.2015.7324181.

[23] Relja Arandjelović et al. *NetVLAD: CNN Architecture for Weakly Supervised Place Recognition*. May 2, 2016. DOI: 10.48550/arXiv.1511.07247. arXiv: 1511.07247 [cs]. URL: http://arxiv.org/abs/1511.07247 (visited on 08/01/2025). Pre-published.

[24] Paul-Edouard Sarlin et al. "From Coarse to Fine: Robust Hierarchical Localization at Large Scale". In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019, pp. 12716–12725.

[25] D Girardeau-Montaut et al. "CHANGE DETECTION ON POINTS CLOUD DATA ACQUIRED WITH A GROUND LASER SCANNER". In: (2005).

[26] K. S. Arun, T. S. Huang, and S. D. Blostein. "Least-Squares Fitting of Two 3-D Point Sets". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-9.5 (Sept. 1987), pp. 698–700. ISSN: 1939-3539. DOI: 10.1109/TPAMI.1987.4767965.

[27] P.J. Besl and Neil D. McKay. "A Method for Registration of 3-D Shapes". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (Feb. 1992), pp. 239–256. ISSN: 1939-3539. DOI: 10.1109/34.121791.

[28] N. Aspert, D. Santa-Cruz, and T. Ebrahimi. "MESH: Measuring Errors between Surfaces Using the Hausdorff Distance". In: *Proceedings. IEEE International Conference on Multimedia and Expo.* . IEEE International Conference on Multimedia and Expo. Vol. 1. Aug. 2002, 705–708 vol.1. DOI: 10.1109/ICME.2002.1035879.

[29] Zihan Lin et al. "Point Cloud Change Detection With Stereo V-SLAM: Dataset, Metrics and Baseline". In: *IEEE Robotics and Automation Letters* 7.4 (Oct. 2022), pp. 12443–12450. ISSN: 2377-3766. DOI: 10.1109/LRA.2022.3219018.

[30] Stefan Larsen et al. "Change Detection and Model Update Framework for Accurate Long-Term Localization". In: ().

[31] María T. Lázaro, Roberto Capobianco, and Giorgio Grisetti. "Efficient Long-term Mapping in Dynamic Environments". In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Oct. 2018, pp. 153–160. DOI: 10.1109/IROS.2018.8594310.

[32] Malcolm Mielle et al. "SLAM Auto-Complete: Completing a Robot Map Using an Emergency Map". In: *2017 IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*. 2017 IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR). Oct. 2017, pp. 35–40. DOI: 10.1109/SSRR.2017.8088137.

[33] Erik Derner et al. "Change Detection Using Weighted Features for Image-Based Localization". In: *Robotics and Autonomous Systems* 135 (Jan. 2021), p. 103676. ISSN: 0921-8890. DOI: 10.1016/j.robot.2020.103676.

[34] Julie Stephany Berrio et al. "Updating the Visibility of a Feature-Based Map for Long-Term Maintenance". In: *2019 IEEE Intelligent Vehicles Symposium (IV)*. 2019 IEEE Intelligent Vehicles Symposium (IV). June 2019, pp. 1173–1179. DOI: 10.1109/IVS.2019.8814189.

[35] Julie Stephany Berrio et al. *Long-Term Map Maintenance Pipeline for Autonomous Vehicles*. Aug. 28, 2020. DOI: 10.48550/arXiv.2008.12449. arXiv: 2008.12449 [cs]. URL: http://arxiv.org/abs/2008.12449 (visited on 07/29/2025). Pre-published.

[36] Pablo F. Alcantarilla et al. "Learning Visibility of Landmarks for Vision-Based Localization". In: *2010 IEEE International Conference on Robotics and Automation*. 2010 IEEE International Conference on Robotics and Automation. May 2010, pp. 4881–4888. DOI: 10.1109/ROBOT.2010.5509383.

[37] Takuya Akiba et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. July 25, 2019. DOI: 10.48550/arXiv.1907.10902. arXiv: 1907.10902 [cs]. URL: http://arxiv.org/abs/1907.10902 (visited on 08/12/2025). Pre-published.

[38] Yair Censor. "Pareto Optimality in Multiobjective Problems". In: *Applied Mathematics and Optimization* 4.1 (Mar. 1, 1977), pp. 41–59. ISSN: 1432-0606. DOI: 10.1007/BF01442131.

Appendices

6.2 Key Algorithm Implementations

Bayesian Update Implementation

```python
def update(self, observation: Observation):
    prior = self.p_exists

    # Model estimation is clamped to [0.001, 0.999] to avoid division by zero or extreme values
    model_estimation = clamp(self.model.predict(observation.viewpoint), 0.001, 0.999)

    if observation.state == ObservationState.SEEN:
        numerator = model_estimation * (1 - self.false_negative_rate) * prior
        denominator = (model_estimation * (1 - self.false_negative_rate) * prior) + (
            self.false_positive_rate * (1 - prior)
        )
        posterior = numerator / denominator

    else:
        numerator = (1 - (model_estimation * (1 - self.false_negative_rate))) * prior
        denominator = (
            (1 - (model_estimation * (1 - self.false_negative_rate))) * prior
        ) + ((1 - self.false_positive_rate) * (1 - prior))
        posterior = numerator / denominator

    if posterior < 0 or posterior > 1:
        raise ValueError(f"Posterior probability out of bounds: {posterior}")

    # If the estimated observability of the point is lower than the actual, we increase the overall observability
    # Otherwise we decrease it
    self.observability = clamp(
        self.observability
        + (
            self.observability_update_factor
            * (observation.state.value - model_estimation)
        ),
        0.25,
        0.75,
    )

```

```python
    # The more observable the point is, the more we dampen the update
    self.p_exists = clamp(
        self.p_exists
        + (posterior - prior) * (1 - self.observability) * self.damping_coefficient,
        0.001,
        0.999,
    )

    self.model.integrate(observation, abs(self.p_exists - prior), self.plot)

    return self.p_exists


```

Listing 1: Python Implementation of the Bayesian Update Step

Full Results